Methodology - Using Learning Automata to Monitor Kelp Forests Adaptively PhD Thesis Research University of Victoria Today

The chapter explains a method for adaptive monitoring of kelp forests. Satellite based environmental monitoring faces a basic conflict - detection algorithms need feedback to become more accurate but feedback arrives only occasionally and at unpredictable moments. We introduce a Learning Automata system that keeps a set of different detection algorithms and learns which one to use according to the surrounding conditions. The system combines three parts - global policy learning, context aware adaptation plus sparse label handling. A key advance is the dual layer non-stationarity model. This model extends classical Learning Automata theory so that it copes with both shifting environmental states and irregular feedback intervals. We test the method - leaving one site out at a time across the coastal kelp forests of British Columbia.

Contents

1 The Difficulty of Watching

British Columbia hosts kelp forests that belong to the planet's most productive ecosystems. Submerged fronds create habitat for myriad organisms, lock carbon into plant tissue and have supplied coastal First Nations with food plus materials for thousands of years. Present day climate shifts, marine heatwaves and unchecked sea urchin grazing impose threats of a scale never recorded. Tracking ecosystem health across entire coastlines forms a critical step toward protection but the work proves unexpectedly hard.

Satellite remote sensing supplies the wide area view that coastline scale monitoring demands. Sentinel-2 and similar sensors record an image of every part of British Columbia's 25 000-kilometre coast once each five days, with each pixel ten metres across. The obstacle is not the capture of the images - it is the conversion of those images into dependable kelp maps. Three linked problems render that conversion intrinsically hard.

1.1 Environmental Heterogeneity

No single detection algorithm performs well in every location. A method that performs accurately in calm summer water will fail if it is used on turbid winter water. Bull kelp that grows on wave exposed outer coasts differs in behaviour from bull kelp that grows in sheltered inland water. Kelp that stands next to a sandy sea floor reflects light differently from kelp that stands on rocky reef. In addition, river plumes, industrial discharge and nearby land create detection problems that are unique to each site.

A deep learning network trained on satellite images of the central British Columbia coast memorises the light response pattern of kelp under that exact setting - the clarity of the water, the kind of seafloor and the usual atmospheric state. When the same network is used on the northern coast, where the water holds different sediments, the depths vary and nearby land cover changes, its accuracy drops. The system has stored only a single set of rules for kelp but each new stretch of coastline demands its own rule set.

1.2 Sparse and Delayed Validation

Collecting ground truth validation data is costly and logistically difficult. Field surveys depend on boats, divers plus suitable weather. Those surveys do not match the speed at which satellites acquire images. A Sentinel-2 image recorded today may wait weeks or months for validation and some images never receive it. The majority of satellite images remain without ground truth labels.

A basic conflict arises - methods that spot problems need information about their past errors so they can become more accurate. But this information appears only now and then and no one knows when the next piece will come. Classic machine learning practice expects plentiful labels that arrive at the same time as the data - the engineer trains the system on a fully labeled set then releases it. When the same system runs in production, it must keep adjusting long after release, fed by the occasional verified example that trickles in.

1.3 Non-Stationary Conditions

The environment and the observation process both shift as time passes. Kelp forests follow a clear seasonal cycle. In spring, the plants grow fast. By summer, the canopy reaches full cover. In fall, the plants age plus decline. Water conditions also shift with the seasons. Winter storms stir up the water, increase turbidity and lower the sun angle. A detection method that works best in August will fail in February.

Over decades, the climate varies and the system flips to a new state. An El NiÃ±o episode raises the temperature of surface seawater and changes how kelp cells function. A sudden multi

week heatwave in the ocean kills a large part of the population. A detection algorithm that was adjusted to the former average situation often gives no useful signal once the anomaly begins.

The process of observation changes over time - field checks follow the seasons - teams carry out most surveys during summer because conditions suit the work. Budget periods, weather setbacks and transport limits shift from one year to the next. Because of those shifts, no fixed schedule governs when feedback reaches the analysts.

1.4 The Core Question

Those challenges relate to one another - environmental heterogeneity forces different methods to suit different settings. Sparse feedback prevents us from training a fresh model for every setting. Non stationarity ensures that even if we trained such a model, the best method would continue to shift.

This raises a question - What if we do not train one model that we call the best but keep a set of different detection methods and learn to choose the right one for each case? What if we find out which method suits which situation, learn this link from limited feedback and change the choice when the situation changes?

We present the exact procedure in this chapter.

The core discovery is that performance improves through selection instead of training.

The usual way to build a machine learning system is to choose the model once and for all. Several models are trained, each is scored on a validation set, the one with the best score is kept plus that single version is put into production. After this point the parameters do not change and the environment is presumed to remain the same.

We introduce a method that differs from common practice - we treat the choice of a model as a task that continues to learn. Instead of locking onto one detection method, we keep a set of varied methods. Each method has distinct advantages and drawbacks. The system learns which method to pick - observing the current surroundings.

1.5 A Committee of Experts

We treat this method as a panel of specialists, where each member trains under a distinct set of conditions. One specialist detects kelp best when the sea is flat and the water transparent. A second specialist maintains accuracy when suspended sediment clouds the scene. A third specialist tolerates illumination that arrives at a low solar elevation. After a new satellite scene downloads, the software inspects the surrounding circumstances - water clarity, solar angle, season and geographic position. The system then appoints the specialist whose training environment most closely matches the current circumstances.

The choice relies on chance, not on certainty - we keep a list of likelihoods for every expert and each value shows how much we trust that expert. When the committee contains three experts, we might now hold the view that Expert A gives a correct result in sixty out of hundred cases, Expert B in thirty besides Expert C in ten. After we receive feedback, which means a verified forecast, we revise those likelihoods. If Expert B surpasses our expectation, its likelihood grows and the remaining values shrink.

As time passes, the probability distribution settles - the system discovers which experts it should rely on for each context. This discovery proceeds without interruption while the system runs and it uses only scarce feedback. No model requires a new training run. The process demands only sporadic validation so that we can refresh the selection preferences.

1.6 Why Learning Automata?

This task, which consists of choosing actions when the feedback is random, is exactly what Learning Automata were created to handle. Learning Automata appeared during the 1960s

as part of control theory and they give a method for reaching decisions in places where the surroundings are not known. One automaton keeps a list of probabilities for every action, picks an action using those probabilities, gets a reward or a penalty then changes the list of probabilities to reflect the result.

Lipid aggregates display a set of attributes that suit our problem.

Sparse feedback tolerance - LA do not need a response after each act. They revise their state only when a response arrives and they stay unchanged when no response arrives.

Non-stationarity handling - LA adapts when the environment changes. After the best action changes, the probability distribution adjusts until it matches the new best action.

The algorithm receives a formal assurance - if the model matches reality in every relevant respect, the sequence of actions that LA produces ends at the action that yields the highest possible reward. This provable property separates LA from rules of thumb that lack any such guarantee.

Simplicity: The update rules demand only a small amount of calculation. They do not need a gradient, they do not need backpropagation - they only adjust probabilities.

We place the full mathematics of LA in Appendix **??**. At this point, the basic idea is enough. In short, LA offer a systematic method to decide which expert deserves trust, using only sparse feedback, even when the environment keeps changing.

Framework Overview

This section offers a concise map of the entire framework - it lists each principal component and shows how data flows from one stage to the next.

This section presents the complete structure of the adaptive monitoring framework. The system ingests satellite image tiles and outputs kelp detection maps. It learns from limited validation feedback plus refines its selection choices as time passes.

1.7 System Architecture

The framework contains three parts that work together - Figure 1 shows their layout.

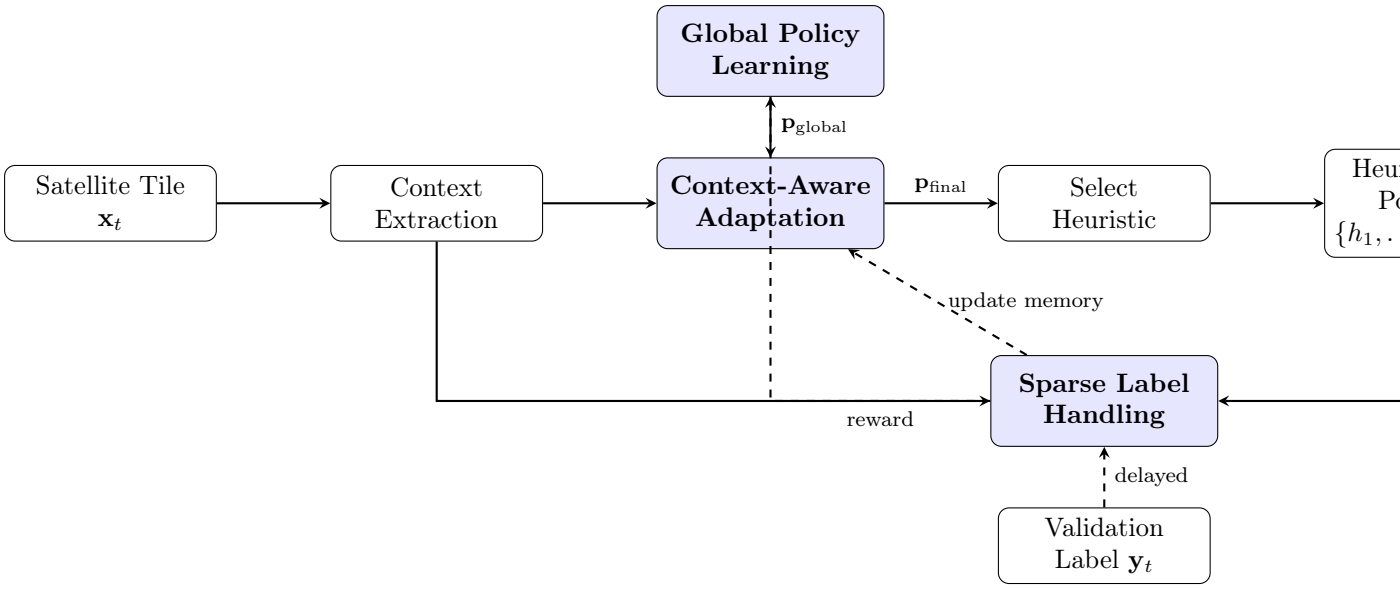

Figure 1: Framework architecture. Solid arrows show the inference path; dashed arrows show feedback paths that operate when validation labels arrive.

Global Policy Learning keeps a probability spread that covers every detection heuristic in the pool. This spread records which heuristics the system has favored in the past because they

delivered good results. Each time feedback arrives, the spread changes through the fixed rules of Learning Automata.

Context-Aware Adaptation reads environmental features from every input tile. It stores earlier encounters in a memory bank and adjusts the global preferences to match the present context. When the system recognises the situation, it relies on previous data. When the situation is new, it returns to the global preferences.

Sparse Label Handling stores predictions until a label appears. After days, weeks or even months, the true label arrives - the system fetches the stored prediction, calculates the reward and then updates Component 1 and Component 2.

1.8 The Inference Path

RESULT$_T OO_L ONG$

Extract context. We calculate environmental attributes from the tile - spectral statistics, texture values, the calendar day and the geographic position. Those attributes combine into a context vector that records the current state.

We begin - searching the memory bank for past situations that resemble the current context vector. When we locate stored contexts whose outcomes were repeatedly favourable, we merge the preferences tied to those old contexts with the parameters of the global policy. If no comparable context exists, we assign almost all weight to the global policy.

Choose a heuristic - we pick a detection heuristic - drawing from a blended probability distribution. The choice is random - heuristics that have a low probability still receive selection at times - exploration continues.

The chosen heuristic scans the tile and outputs a binary mask that labels each pixel as either kelp or no kelp.

The prediction, the context vector and the index of the chosen heuristic are all placed in a buffer so they can be checked later.

1.9 The Feedback Path

A validation label reaches us weeks after the first prediction. We handle it in this way:

Fetch the stored forecast. We use the position and the exact time to find the matching record in the buffer.

Compute the reward - we assess how closely the prediction aligns with the ground truth, usually - calculating the Intersection over Union (IoU) metric.

The global policy receives an update - the reward signal alters the probability distribution that spans the heuristics. When a heuristic performs well, its probability rises - when it performs poorly, its probability falls or it stays the same, according to the rules of the LA scheme.

Refresh the memory bank. We record the context vector and the performance that we observe - later adaptation can use that context.

A central point is that those feedback updates operate apart from inference. The system keeps producing predictions and does not pause until a label shows up. Once the label arrives, the system adjusts its parameters. The adjustment happens at a different time from the prediction, occurs only now plus then and lags behind the original input.

Component 1: Discovering Effective Methods Label - sec:component1

The first component stores the system's general preferences among detection heuristics. It records the system's broad view on which methods usually succeed, computed as an average across every context the system has processed.

1.10 The Probability Distribution

The global policy is a probability distribution over r heuristics:

$$\mathbf{p} = [p_1, p_2, \ldots, p_r]$$

where each p_i represents the system's confidence in heuristic h_i. Initially, the distribution is uniform; the system has no prior preference:

$$p_i(0) = \frac{1}{r} \quad \text{for all } i$$

Feedback grows over time and the distribution changes - heuristics that achieve strong results gain probability mass. Heuristics that deliver weak results lose probability mass. In the end, the distribution centers on the heuristics that perform best.

1.11 The Update Rule

A prediction that heuristic h_i produces receives validation. At that moment the system measures detection accuracy and assigns a reward. The reward R equals 0 when the detection fails entirely plus equals 1 when the detection succeeds without error.

The probability revision relies on the Learning Automata rule that is chosen. The most basic rule, called Linear Reward Inaction ($\mathrm{L_{R\text{-}I}}$), revises the probability only when positive feedback arrives.

- When the reward rises above the threshold that marks good performance, raise the value of p_i and lower the remaining values in proportion to their current size.
- When the reward stays below the threshold that indicates poor performance, leave every value unchanged.

This design, which refrains from imposing an immediate penalty, equips the system with tolerance to noisy data. One substandard outcome does not instantly penalize a heuristic - only repeated high quality results receive a reward. Formal update equations appear in Appendix **??**.

Other approaches - Linear Reward Penalty, Pursuit besides Variable Structure - balance learning speed against stability in different ways. We test six versions, each outlined in Appendix **??** and we determine the most effective ones for kelp detection by means of ablation studies.

1.12 Why This Works

A global policy acts as the system's preset preference - when no context specific data exist, it supplies the most reliable estimate that is currently available. Its value comes from pooling results that were gathered in every context - if heuristic h_3 yields a good outcome in seventy percent of varied situations, the system records this fact - assigning a high value to p_3.

Global preferences operate as blunt tools - a heuristic that performs well during summer often fails in winter. The global policy smooths out those seasonal differences. A second component becomes necessary to capture patterns that depend on specific contexts.

Component 2: Adjustment to Context

The second component lets the system change which heuristic it selects when the environment around it changes. The idea is simple - if the present situation looks like an earlier situation in which one heuristic performed well, the system gives preference to that same heuristic.

1.13 Context Features

Each satellite tile contains data that describes the environmental situation that existed when the image was taken. We isolate a context vector c that stores:

Each spectral band supplies its mean and its variance - those two numbers record water colour, turbidity plus atmospheric state.

Entropy and homogeneity values describe spatial texture - they reveal surface roughness but also wave condition.

The calendar day provides temporal context - it encodes seasonal cycles.

Latitude and longitude give geographic position - they encode attributes that vary from region to region.

Bathymetry as well as substrate type serve as auxiliary data - they are used only when measurements exist.

The context vector holds a summary of the conditions that apply. When two tiles have context vectors that are close, they probably need the same detection strategy.

1.14 The Memory Bank

We store a collection of past events - every record holds three items - a context vector, the set of heuristic probabilities that were active in that situation and the reward that followed. As time passes, the collection grows plus preserves a clear trace of which actions succeeded under which conditions.

A fresh tile brings its context vector c - we look up the k earlier contexts that lie closest to c by exact nearest neighbor search. To keep the lookup fast while the memory bank grows, we run the search through FAISS, a library that performs approximate nearest neighbor search with high speed.

1.15 Local Preference Estimation

The system fetches neighboring cases that serve as evidence about effective choices under comparable conditions. It builds a local preference distribution through a weighted average of the neighbors' preferences, where the weights derive from both the rewards those neighbors obtained and their measured similarity to the current case.

Neighbors that show strong similarity to the present context and that earlier obtained high rewards receive larger weight when the agent forms its local estimate. The procedure outputs a distribution $\mathbf{p}_{\text{local}}$ that records which actions succeeded under comparable conditions.

1.16 Blending Global and Local

The final selection probability blends global and local preferences:

$$\mathbf{p}_{\text{final}} = \alpha \cdot \mathbf{p}_{\text{global}} + (1 - \alpha) \cdot \mathbf{p}_{\text{local}} \tag{1}$$

The blending coefficient α sets the balance.

If α equals 1, the system relies solely on the global policy and disregards local context.

If α equals 0, the system relies solely on local experience plus disregards global trends.

For values between 0 and 1, the system combines information from both sources.

We adjust α according to the contents of the memory bank. If the bank stores many near identical contexts whose outcomes agree, α drops and the system trusts local experience. If the contexts are new or their outcomes conflict, α rises plus the system relies on global preferences.

1.17 Why This Works

Context-aware adaptation uses patterns that already exist in the problem. The surrounding conditions do not appear in a random order - they form groups. When the water in the Strait of Georgia is turbid in winter, that state resembles other winter turbid states in the same place. If a heuristic works under such a state, it will probably work the next time the same state appears.

The memory bank stores past solutions so that the system can reuse them without learning new parameters. No model learns to estimate heuristic quality from context descriptors. The system records successful cases and fetches them once a matching situation appears. This method resists disturbance, allows clear explanation plus demands no extra training phase.

RESULT$_T OO_L ONG$

The third component deals with the practical fact that validation labels appear only now and then and arrive after unpredictable delays. The majority of predictions will never receive a label. When a label does arrive, it may come days, weeks or months after the original prediction. The system has to keep running without pausing for this feedback but it must still adjust its model once the feedback finally appears.

1.18 The Prediction Buffer

Each forecast is stored in a buffer - the buffer is a set of ordered records

$$\mathcal{B} = \{(t_j, \mathbf{c}_j, i_j, \hat{\mathbf{y}}_j)\}$$

Each record holds the exact moment of the forecast, the context vector that was active, the index of the heuristic that was chosen and the mask that was predicted. A hash map gives rapid access to any record when the query specifies both spatial coordinates and a time value.

A validation label reaches us for a specific location and date. We fetch the stored prediction that matches that label, calculate the reward plus start updates for Component 1 and Component 2. After that, we delete the entry from the buffer.

1.19 Buffer Management

A system does not allow a forecast to remain valid for an unlimited period. External factors shift - a forecast issued six months earlier may fail to match the data that the system now studies. To prevent stale guidance, the code stores each forecast with a time stamp and enforces a strict upper limit on storage duration, for example six months. Any forecast whose age exceeds this limit is deleted and no attempt is made to revise it.

Our design philosophy is that the system must learn from feedback that arrives soon after an event, not from feedback that arrives long after conditions have changed. A reward that the system receives at once carries more useful information than a reward that reaches the system once the situation is already different.

1.20 Temporal Decay

The system reduces a reward when the validation occurs after a delay. The adjusted reward equals the original reward multiplied by the decay factor raised to the power of the delay counted in days. Recent feedback keeps its full value - earlier feedback loses value.

This decay fulfils two roles

1. **Recency takes priority:** Experience that arrived recently has higher relevance to present conditions.

2. **Non-stationarity is tracked:** When the optimal heuristic shifts through time, old rewards lose weight and the system adjusts.

1.21 Learning Without Waiting

The central design rule is asynchronous learning - the inference path, which takes a tile as input and produces a prediction as output, runs without pause plus does not wait for validation. The feedback path, which receives a label as input and sends an update as output, runs whenever a label becomes available. Those two paths do not depend on one another.

This design fits the way environmental monitoring works in practice. A satellite records an image once every multiple days. A field team performs validation only when funds, staff and weather allow. The system does not pause until validation arrives - it issues forecasts right away. Each new validation result enters the training set - the model updates its parameters.

2 Fresh Element - Two Tier Non-Stationarity

Classical Learning Automata theory studies systems that must pick actions in situations where the best choice shifts repeatedly. The present study adds a further source of change - the moment at which information about the best action becomes available also varies.

2.1 Two Sources of Change

Kelp forest monitoring confronts two separate causes of non stationarity that analysts must treat at the same time:

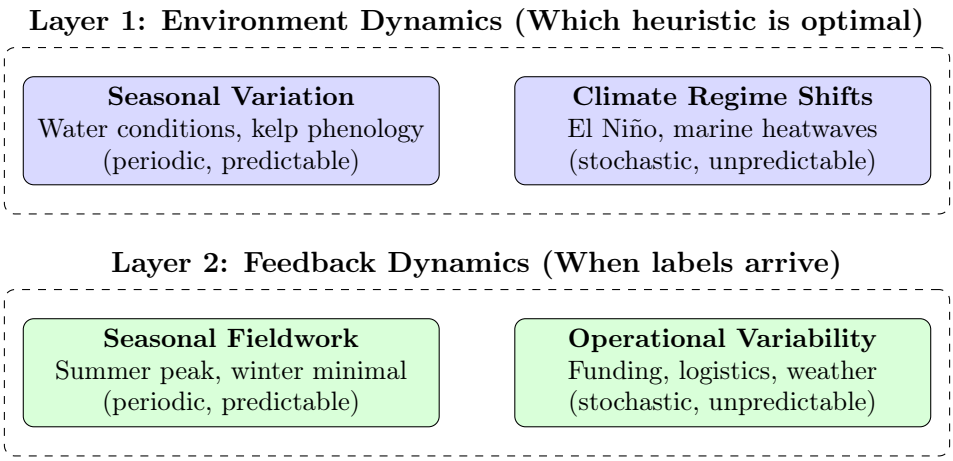

Figure 2: Dual-layer non-stationarity model. Layer 1 governs which detection method is optimal. Layer 2 governs when validation feedback is available. Both layers combine periodic (seasonal) and stochastic (inter-annual) components.

2.1.1 Layer 1: Environment Dynamics

The optimal detection heuristic evolves over time - two mechanisms cause this change.

Kelp forests repeat the same sequence each year - during summer, the surface layer of fronds reaches full size and satellites spot it without difficulty. After winter storms arrive, waves break off large sections of the canopy and stirred up particles cloud the water - the remaining cover is harder to see. A rule that works in clear summer water often gives poor results once turbidity rises. The best rule changes in a regular way that matches the calendar.

Climate regime shifts (stochastic) Year-to-year variability brings sudden, unpredictable changes. El NiÃ±o episodes raise or lower sea temperatures and disrupt kelp metabolism. Marine heatwaves create extremes that have no earlier match. Each upheaval favors a different heuristic but the sequence of upheavals has no fixed rhythm.

In classical learning automata theory, researchers label the first type Periodic Switching Environment and the second type Markovian Switching Environment. Layer 1 unites the two approaches.

2.1.2 Layer 2: Feedback Dynamics

This layer represents the moment when validation labels reach the system. It extends LA theory in a new way. Feedback timing depends on two mechanisms.

Many surveys that check data take place in summer, because calm weather and gentle seas allow teams to work from small boats. Surveys in winter almost never occur. The chance that new data arrives at the office rises plus falls with the seasons.

The amount of money that supports research changes from year to year. The number of trained field staff who are ready to work also rises and falls. Snow, wind or broken instruments force sudden pauses. In some seasons the team gathers many validation records - in other seasons the team gathers few.

Layer 2 stands apart from Layer 1 - on a summer day, Layer 1 may offer ideal detection conditions, while Layer 2 shows a high chance of feedback. On a winter day, Layer 1 may present difficult detection conditions, while Layer 2 shows a low chance of feedback. The states of the two layers do not have to match.

2.2 Why This Matters

Classical LA theory addresses Layer 1, where the environment shifts and the automaton must follow the best action as it changes. This theory takes for granted that feedback will appear at steady intervals. In real systems, feedback appears only when its own rules allow it.

The dual layer model captures this reality

- **The system receives no feedback for long stretches.** During winter, multiple months often pass without any validation. The system must continue to operate in a stable manner and must do so without fresh data.

  The system receives sudden surges of feedback - during summer, multiple surveys confirm forecasts in quick succession. The system needs to absorb this feedback without delay.

  The best action and the chance to receive feedback often move in opposite directions. When winter conditions arrive plus certain heuristics perform well, the system receives sparse validation data. It therefore gains little knowledge about the very conditions where it most needs to adapt.

  By modeling both layers explicitly, the team analyzes learning dynamics under realistic conditions. The team designs LA schemes that stay stable during droughts and stay responsive during bursts. The team tests robustness against different feedback patterns.

2.3 Scientific Contribution

The dual layer non-stationarity model offers three distinct advances.

A new theoretical extension - We expand the standard learning automata theory that deals with environments that change over time. We treat the moment when feedback arrives as a second random sequence. This step raises new analytical questions - How much does learning worsen when feedback arrives only rarely? Which learning automata designs stay most stable when the arrival time of feedback varies?

- **Domain-appropriate modeling:** The formulation records the true attributes of environmental monitoring. Seasonal patterns and operational changes exist. When the model states them, it becomes more realistic.

  Ablation capability - The system allows each layer to be set on its own. Layer 1 controls environment dynamics - Layer 2 remains fixed while Layer 1 changes and the reverse is also possible. Tests carried out in this way reveal how each source of non stationarity alters learning.

  Section 6 explains the plan that will be used to judge how well the method works.

  We tested the framework with photographs of kelp forests taken along the coast of British Columbia. The evaluation relied on Leave-One-Site-Out cross validation and followed a staged ablation protocol.

2.4 Datasets

Table 1: Datasets used for evaluation

Dataset	Images	Labels	Purpose
BC Sentinel-2 (Labeled)	7	Yes	Primary training and validation
BC Sentinel-2 (Multi-temporal)	∼100	No	Layer 1 testing, sparse simulation
Figshare Landsat	432	Yes	Cross-sensor transfer

We built the main dataset from Sentinel-2 images that cover seven locations along the coast of British Columbia. Specialists drew precise outlines of kelp for every image. Each tile measures 512 by 512 pixels and every pixel represents ten metres on the ground. The tiles store twelve channels - spectral layers plus auxiliary data. Complete details of the dataset appear in Appendix A.

2.5 Heuristic Pool

The heuristic pool contains two groups of models.

Ten site specific models exist - each model uses a UNet-MaxViT architecture and trains under the Leave-One-Site-Out protocol. Every model trains on data from nine sites and excludes one site - each model experiences a distinct training set.

One RGB model exists - this model trains on all BC sites but limits input to the visual spectral bands. The restriction to visible wavelengths allows the model to transfer to Landsat imagery, because Landsat records the same three bands.

2.6 Leave-One-Site-Out Protocol

For the main test, we exclude one site from the set - we train the LA system on the remaining six sites. We then test the trained system on the excluded site. We repeat this process for each of the seven sites. We combine all individual test outcomes into a single summary.

The protocol evaluates how well a system performs when it encounters geographic regions it has not seen before. This ability is essential for deployment across an entire coastline.

2.7 Staged Ablation

A complete test of every design option would demand tens of thousands of trials. To keep the experimental plan feasible, we use a stepwise method:

Stage 1: Selection of the LA variant - we test six LA schemes - L_{R-I}, L_{R-P}, VSLA, Pursuit, Discretized Pursuit besides SERI - against the complete system. We then select the two or three schemes that perform best.

We test combinations of components - global only, global plus context, global plus sparse and the full system - using the top LA variants.

Stage 3: We test how the system tolerates sparse labels - we lower the proportion of labeled data stepwise to 100 percent, 50 percent, 10 percent and 5 percent while we keep the delay pattern that gave the best result in the previous stage.

Stage 4: Auxiliary data - we conduct tests that either include or exclude bathymetry, substrate and additional auxiliary layers.

Stage 5: We test the RGB-trained model on Figshare Landsat data that come from a different sensor.

This step-by-step method lowers the number of required computations from more than 46 000 to about 500 and it still upholds scientific standards.

2.8 Baselines

We compare our method with three reference strategies. We include a baseline that selects a heuristic at random, each option chosen with equal probability. We test against the single fixed heuristic that achieves the highest average score across the entire dataset. We use an oracle that, for every tile, picks the heuristic that yields the best possible result - this sets an upper bound that would demand perfect advance knowledge.

2.9 Metrics

Primary detection metrics

Intersection over Union (IoU): This value shows how much the predicted kelp mask aligns with the actual kelp mask. It equals the area where both masks agree divided by the total area that either mask claims.

- F1-score - This single number balances precision and recall. It equals two times their product divided by their sum.

Los-Angeles-specific metrics - Convergence speed - the number of steps the system needs until the performance value reaches ninety percent of its final level. - Final entropy - the degree to which the probability distribution concentrates when the system has converged.

We present exact definitions in Appendix **??**.

Section - Summary Label - sec:summary

This chapter describes a method for monitoring kelp forests that adjusts to changing conditions. The approach solves basic problems that occur when researchers use satellites to observe the environment.

The core insight treats detection as a selection task instead of a training task. We keep a collection of varied methods and learn to pick one according to context, rather than build a single model that must function in every situation.

The framework consists of three components. Global policy learning monitors the overall success of heuristics through Learning Automata. Context aware adaptation adjusts the selection of heuristics to match environmental conditions - retrieving information from memory. Sparse label handling supports learning when validation feedback arrives irregularly or with delay.

The new element is a dual layer non-stationarity model - it enlarges classical LA theory so that the system copes with two problems - the environment alters over time (Layer 1) and feedback arrives at irregular moments (Layer 2). The model gives a theoretical basis for learning when an operational monitor has to work under those two practical limits.

Evaluation applies Leave-One-Site-Out cross validation across kelp forests located along the coast of British Columbia. The process includes staged ablations that isolate effective configurations and assess robustness when feedback is sparse.

Later chapters set out the experimental outcomes that confirm every component and that show how well the framework performs when it monitors kelp forests in an adaptive way.

Section - Formal Problem Definition Label - app:formal

.1 Input Space

A satellite image captured at time t is stored as a three dimensional array $\mathrm{x}_t. Its first axis spans the 12 spectr$
$2, the second axis spans 512 rows and the third axis spans 512 columns - each pixel covers a ground square of sid$
$the tile measures 5.12 km 5.12 km.$

The context vector $\mathbf{c}_t \in \mathbb{R}^D$ encodes environmental conditions:

$$\mathbf{c}_t = \left[\mathbf{c}_t^{\text{spectral}}, \mathbf{c}_t^{\text{spatial}}, \mathbf{c}_t^{\text{temporal}}, \mathbf{c}_t^{\text{environ}}, \mathbf{c}_t^{\text{geo}}\right]^\top \tag{2}$$

.2 Heuristic Pool

The heuristic pool $\mathcal{H} = \{h_1, \ldots, h_r\}$ contains r detection methods, each mapping tiles to binary masks:

$$h_i : \mathbb{R}^{B \times H \times W} \to \{0, 1\}^{H \times W} \tag{3}$$

.3 Reward Function

The primary reward is Intersection over Union:

$$R(\hat{\mathbf{y}}, \mathbf{y}) = \mathrm{IoU}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{|\hat{\mathbf{y}} \cap \mathbf{y}|}{|\hat{\mathbf{y}} \cup \mathbf{y}|} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}} \tag{4}$$

.4 Policy Objective

We learn a policy $\pi : (\mathbf{x}, \mathbf{c}) \to \Delta^{r-1}$ maximizing expected reward:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{(\mathbf{x},\mathbf{c},\mathbf{y})\sim\mathcal{D}} \left[\sum_{i=1}^{r} \pi(\mathbf{x}, \mathbf{c})_i \cdot R(h_i(\mathbf{x}), \mathbf{y})\right] \tag{5}$$

Section - Basic Ideas of Learning Automata

.5 Basic Formulation

A Learning Automaton consists of four fixed parts - a set of actions, a set of responses, a vector of probabilities that changes over time and a rule that alters the vector. The set of actions contains every distinct choice the automaton may select - each action is labeled Î±â through Î±áµ£. The set of responses contains only two symbols: 0 stands for reward and 1 stands for penalty. At every step n, the automaton holds a probability vector p(n) - each entry páµ¢(n) gives the chance that action Î±áµ¢ will be chosen next. The rule T revises the entire vector after each action and its resulting response - the probabilities shift as experience accumulates.

The environment contains a set of penalty probabilities, each denoted by c_1 through c_r. For every c_i, the value equals the conditional probability that the random variable β takes the value 0 given that the random variable α equals the specific outcome α_i.

RESULT$_T OO_L ONG$

Update on reward only:

$$\begin{aligned} \text{If } \beta(n) = 1: \quad & p_i(n+1) = p_i(n) + \alpha(1 - p_i(n)) \quad \text{(selected action)} && (6) \\ & p_j(n+1) = p_j(n)(1-\alpha) \quad j \neq i && (7) \\ \text{If } \beta(n) = 0: \quad & p_i(n+1) = p_i(n) \quad \forall i && (8) \end{aligned}$$

If the penalty probabilities are all different and ordered as câ < câ < â¦ < cáµ£, the L$_{R-I} schemeselectsthebestact$

This subsection introduces the Linear Reward Penalty model, abbreviated as L$_{\text{R-P}}$.

Update on both reward and penalty:

$$\begin{aligned} \text{If } \beta(n) = 1: \quad & p_i(n+1) = p_i(n) + \alpha(1 - p_i(n)) && (9) \\ & p_j(n+1) = p_j(n)(1-\alpha) \quad j \neq i && (10) \\ \text{If } \beta(n) = 0: \quad & p_i(n+1) = p_i(n)(1-\beta) && (11) \\ & p_j(n+1) = p_j(n) + \frac{\beta}{r-1}(1 - p_j(n)) \quad j \neq i && (12) \end{aligned}$$

.6 Pursuit Automata

We maintain reward estimates $\hat{d}_i(n)$ and pursue the current best:

$$\hat{d}_i(n) = \frac{\text{Total rewards for } \alpha_i}{\text{Total selections of } \alpha_i} \tag{13}$$

$$m(n) = \arg\max_i \hat{d}_i(n) \tag{14}$$

$$p_{m(n)}(n+1) = p_{m(n)}(n) + \alpha(1 - p_{m(n)}(n)) \tag{15}$$

.7 Variable Structure LA (VSLA)

Adaptive learning rate based on entropy:

$$H(\mathbf{p}) = -\sum_{i=1}^{r} p_i \log_2 p_i \tag{16}$$

$$\alpha(n) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot \frac{H(\mathbf{p}(n))}{\log_2 r} \tag{17}$$

RESULT$_T OO_L ONG$

Bayesian estimation uses a Beta distribution as the posterior for every parameter Î¸i. The formula states that the posterior density of Î¸i given the observed data is proportional to a Beta distribution whose shape parameters are Î±i and Î²i. The procedure applies pursuit style update rules. The resulting scheme belongs to the fastest algorithms that deliver an Îµ-optimal solution.

Section - Algorithm Pseudocode Label - app:algorithms

Algorithm 1 Framework Inference

Require: Tile $\mathbf{x}$, global policy $\mathbf{p}_{\text{global}}$, memory bank $\mathcal{M}$

$\mathbf{c} \leftarrow \text{ExtractContext}(\mathbf{x})$
$\mathcal{N}_k \leftarrow \text{kNN}(\mathbf{c}, \mathcal{M}, k)$
$\mathbf{p}_{\text{local}} \leftarrow \text{WeightedAverage}(\mathcal{N}_k)$
$\alpha \leftarrow \text{ComputeBlendingCoef}(\mathcal{N}_k)$
$\mathbf{p}_{\text{final}} \leftarrow \alpha \cdot \mathbf{p}_{\text{global}} + (1 - \alpha) \cdot \mathbf{p}_{\text{local}}$
$i \sim \text{Categorical}(\mathbf{p}_{\text{final}})$
$\hat{\mathbf{y}} \leftarrow h_i(\mathbf{x})$
$\mathcal{B} \leftarrow \mathcal{B} \cup \{(t, \mathbf{c}, i, \hat{\mathbf{y}})\}$ ▷ Buffer prediction
return $\hat{\mathbf{y}}$

Algorithm 2 Label Arrival Processing

Require: Validation label $\mathbf{y}$ for location ℓ, time t

$(t_j, \mathbf{c}_j, i_j, \hat{\mathbf{y}}_j) \leftarrow \mathcal{B}[\text{lookup}(\ell, t)]$
$\delta \leftarrow t_{\text{current}} - t_j$ ▷ Delay
if $\delta > T_{\max}$ **then**
    Remove from buffer; **return** ▷ Too old
end if
$R \leftarrow \text{IoU}(\hat{\mathbf{y}}_j, \mathbf{y}) \cdot \gamma^{\delta}$ ▷ Reward with decay
$\mathbf{p}_{\text{global}} \leftarrow \text{LAUpdate}(\mathbf{p}_{\text{global}}, i_j, R)$ ▷ Update Component 1
$\mathcal{M} \leftarrow \mathcal{M} \cup \{(\mathbf{c}_j, \mathbf{p}_{\text{final}}, R)\}$ ▷ Update Component 2
Remove entry from $\mathcal{B}$

A Dataset Details

A.1 BC Sentinel-2 Dataset

Table 2: BC Sentinel-2 sites

Site	**UTM**	**Date**	**Season**	**Tiles**
T09UXQ	9U	July 27, 2021	Summer	∼40
T09UYQ	9U	August 6, 2022	Summer	∼45
T10UCU	10U	August 6, 2022	Summer	∼50
T09UXS	9U	August 4, 2023	Summer	∼35
T10UCA	10U	August 16, 2023	Summer	∼40
T09UWT	9U	August 19, 2023	Summer	∼45
T09UUU	9U	September 2, 2023	Summer	∼42
Total				∼300

A.2 Band Specification

Table 3: 12-band tile structure

Band	Source	Resolution
1–4	B2, B3, B4, B8	10m (native)
5–10	B5, B6, B7, B8A, B11, B12	10m (resampled from 20m)
11	Substrate classification	10m (resampled)
12	Bathymetry	10m (resampled)

B Metrics Definitions

B.1 Detection Metrics

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \tag{18}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{19}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{20}$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{21}$$

B.2 LA Metrics

$$\text{Entropy} = H(\mathbf{p}) = -\sum_{i=1}^{r} p_i \log_2 p_i \tag{22}$$

$$\text{Convergence} = \min\{n : \text{IoU}(n) \geq 0.9 \cdot \text{IoU}_{\text{final}}\} \tag{23}$$

The bibliography section begins. We select the plain style for the list of references. The list environment opens and reserves space for up to ninety nine entries.

K. S. Narendra besides K. S - thathachar wrote a book titled Learning Automata - An Introduction, which Prentice Hall published in 1989.

M. A. L. Thathachar besides P. S - sastry present a survey of the main classes of learning automata. The article appears in IEEE Transactions on Systems, Man or Cybernetics, Part B, volume 32, issue 6, pages 711 - 722, published in 2002.

S - b., Dupont, C., Boyer, L., Juanes, F. and Costa, M. Passive remote sensing technology for mapping bull kelp. Global Ecology besides Conservation, 22:e00948, 2020.

S. and co authors - heterogeneity within British Columbia kelp forest communities. Journal of Phycology, 2024.

A. Mora Soto and co authors identify the principal obstacles that hinder the reliable identification of kelp forests across the entire planet in a 2024 paper for Remote Sensing of Environment.